CSC 413 Project Documentation
Summer 2022

Saru Shrestha
#921987206
Class Section 01

GitHub Repository Link:
https://github.com/csc413-SFSU-Souza/csc413-tankgame-saru-s.git

Table of Contents

PROJECT INFORMATION **3**

INTRODUCTION OF THE TANK GAME **3**

SCOPE OF WORK **3**

BACKGROUND / GIVEN RESOURCES **4**

- Development environment 4
- How to build or import the game in the IDE 4
- How to run your game. 14

ASSUMPTION **15**

TANK WARS GAME CLASS DIAGRAM **16**

FILE STRUCTURE OF THE PROGRAM **17**

DESCRIPTION OF TANK WARS GAME SPECIFIC CLASSES **18**

- Launcher Class 18
- StartGamePanel Class 18
- EndGamePanel Class 18
- GameConstants Class 19
- Resources Class 19
- Collidable Interface 19
- GameObject Class 20
- Tank Class 20
- TankControl Class 20
- Wall Class 20
- Breakable Class 21
- Bullet Class 21
- SpeedPowerUp Class 21
- HealthPowerUp Class 22
- LivesPowerUp Class 22
- GameWorld Class 22

SELF-REFLECTION ON SOFTWARE DESIGN, DEVELOPMENT AND REUSABILITY **23**

CONCLUSION **23**

Project Information

This project is a software development project developed using Computer Programming Language Java. The game is built into a JAVA Archives (JAR). The reason for using JAR is to combine Java class files and associated metadata and resources (such as images, animations, sound effects) into one file which makes it easy to distribute application software or libraries on the Java platform. The selected concept for the software development is game. This project illustrates the game development of *Tank Wars.*

Introduction of the Tank Game

This project is a software development assignment demonstrating game development. The chosen game development is *Tank Wars*. *Tank Wars* is a gaming world where the tanks battle against each other. This game is two players game where each player controls each tank. The players fight against each other. Check point such as health points is provided. Health points keep track of each tank. When the tank receive attack from the opponent tank, the health points decline. When the tank does not have any more health and lives attribute available, the tank loses, and the opponent tank wins the game.

Scope of Work

Project objectives:

To build a *Tank Wars* game in order to showcase our software development skills.

Resources available:

Lectures from professor (in-class and videos)

Java IDE

Java starter code

Deliverables:

Tank Wars game

Project roadmap and timeline:

July 7, 2022 – Tank Wars Game Diagram

August 4, 2022 – Implement Tank Wars Game

August 5, 2022 – Documentation

Out of scope:

Limited software development experience

Short period of time

Background / Given Resources

Development environment

The program was built using Java version “18.0.1.1” on IntelliJ IDEA as JAVA IDE.

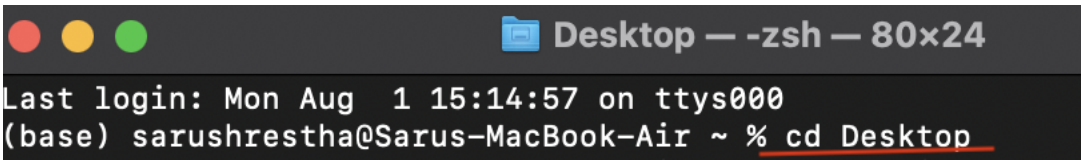
In addition, Java Utility JAR files are used as library to build the program.

How to build or import the game in the IDE

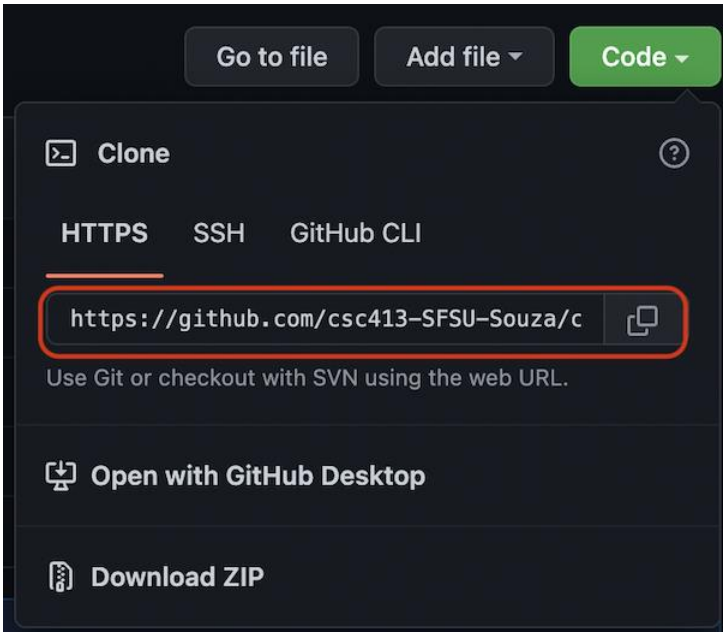
To import the game in the IDE:

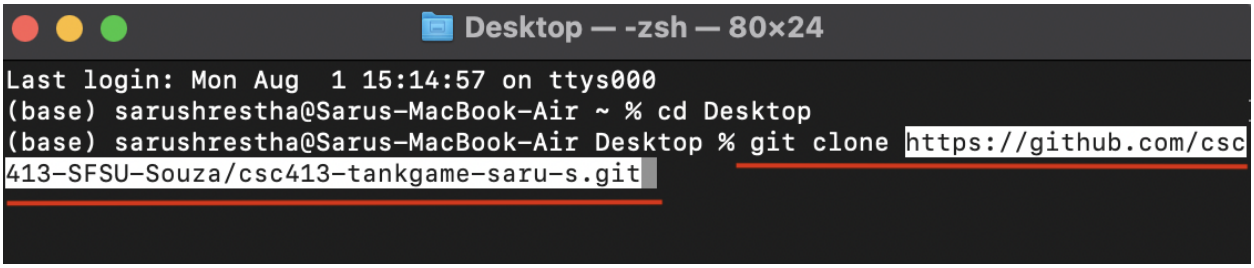
First, open terminal and set the directory to where you desire to import the game.

In this case, I set the directory to Desktop.

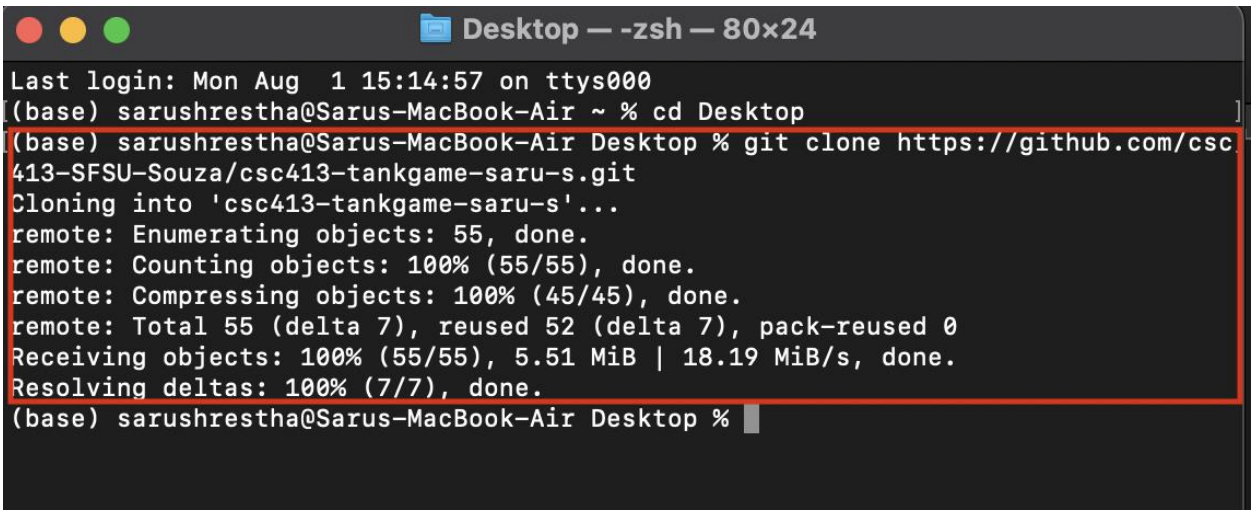


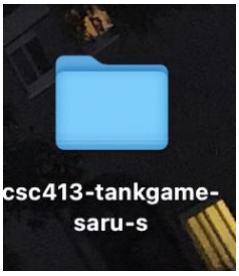
Then clone the git repository by copying the URL of the repository.



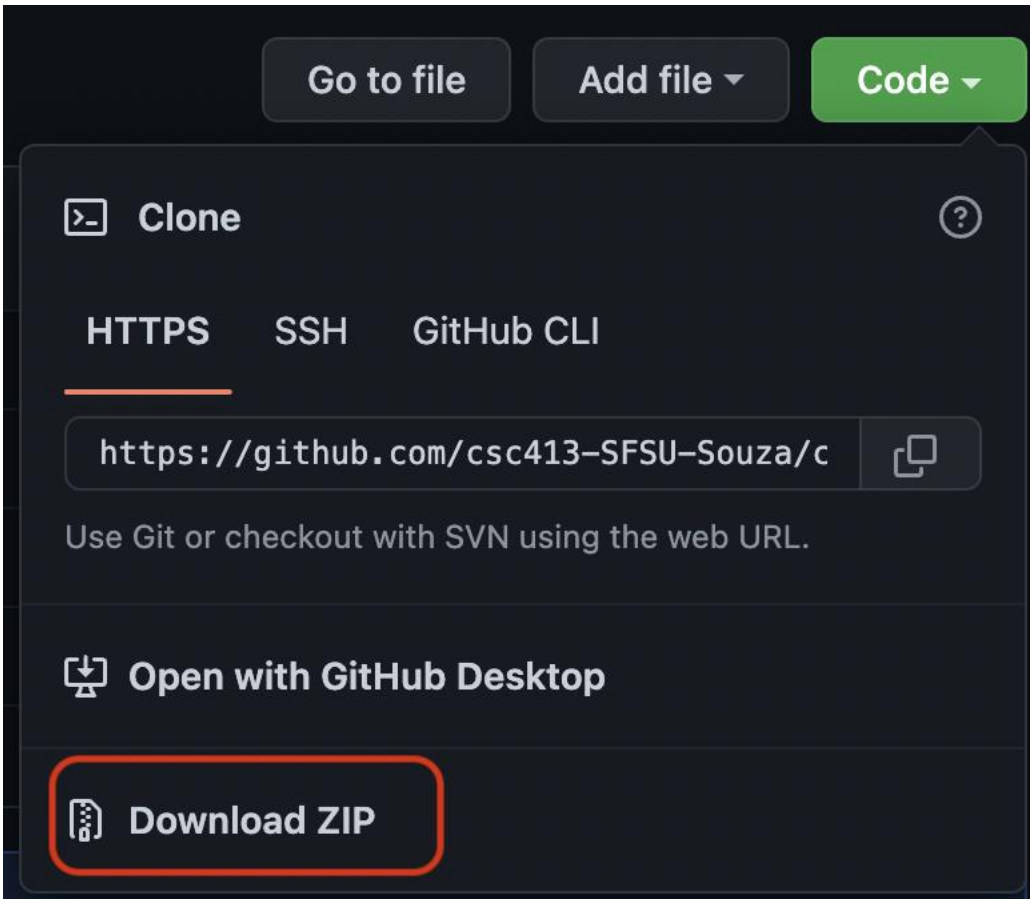


When the cloning is completed, the update can be seen in the terminal. Then, the file is added in the chosen space.

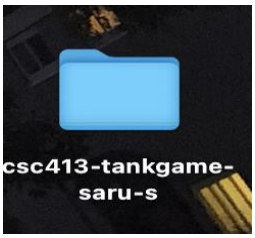




The alternative way to import the file is downloading the zip file from the git repository.



After downloading, when we click the compressed zip file, we get the unzipped file.

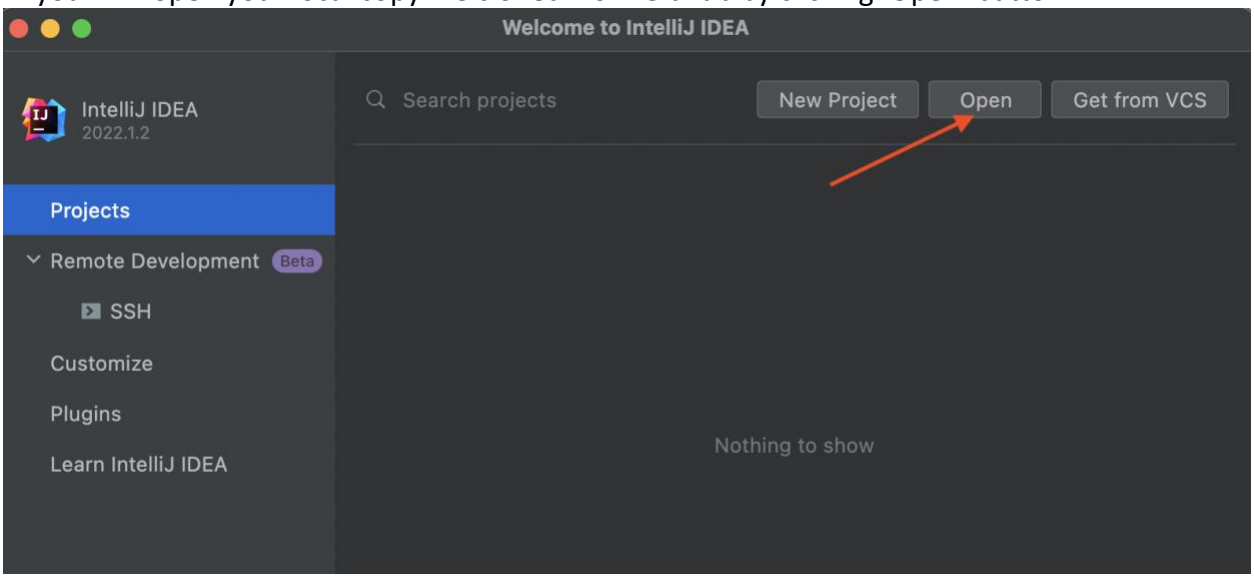


After importing the game in the local computer, we build it in following ways:

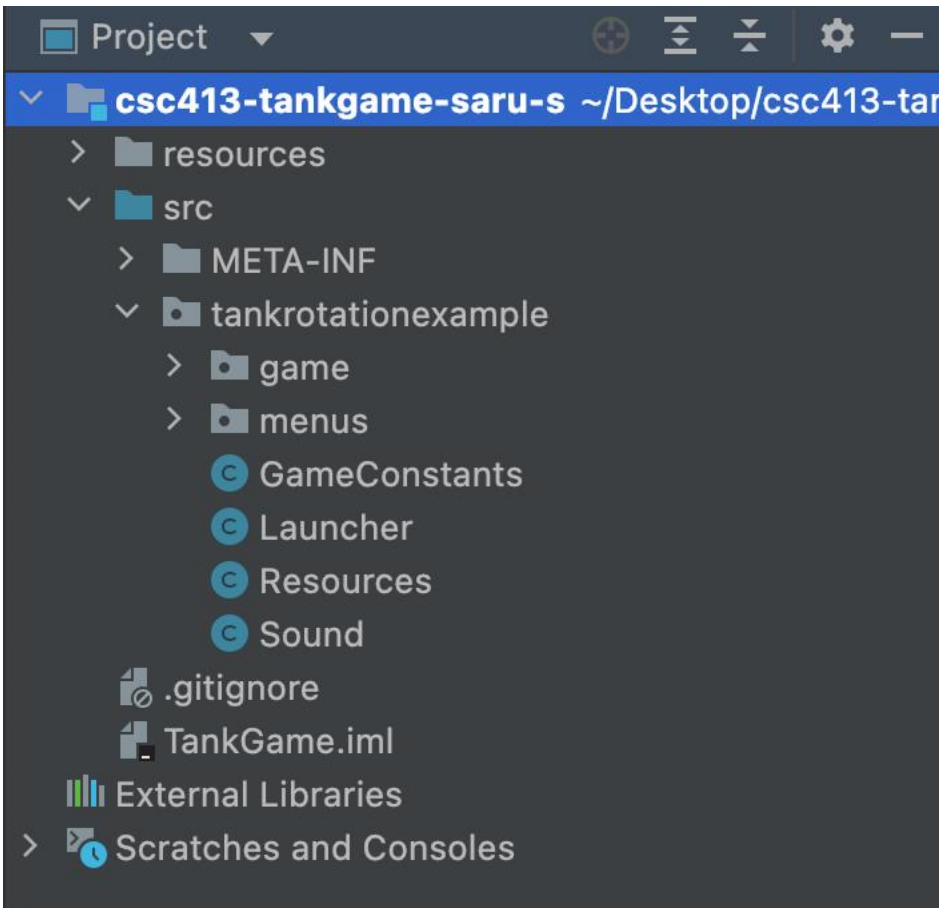
Open the file in the preferred JAVA IDE. In this case, IntelliJ IDEA is used.



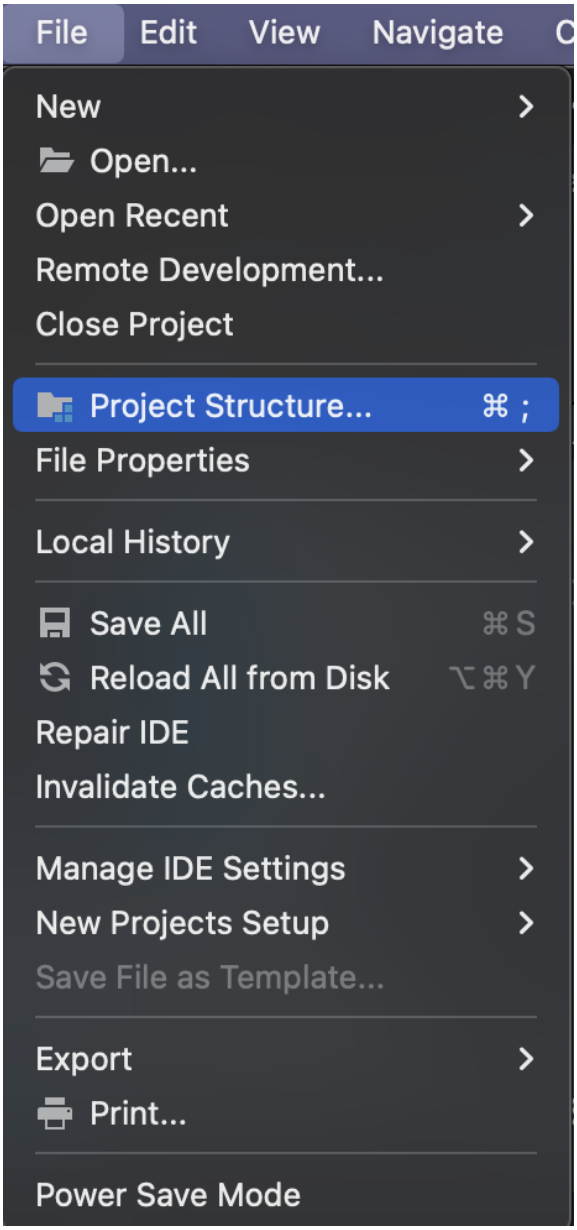
In your IDE open your local copy file cloned from GitHub by clicking ‘Open’ button.

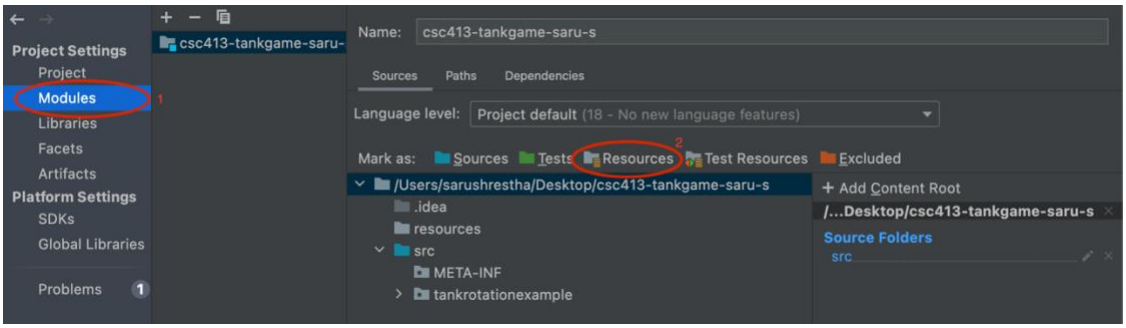


The file structure would be as follows:

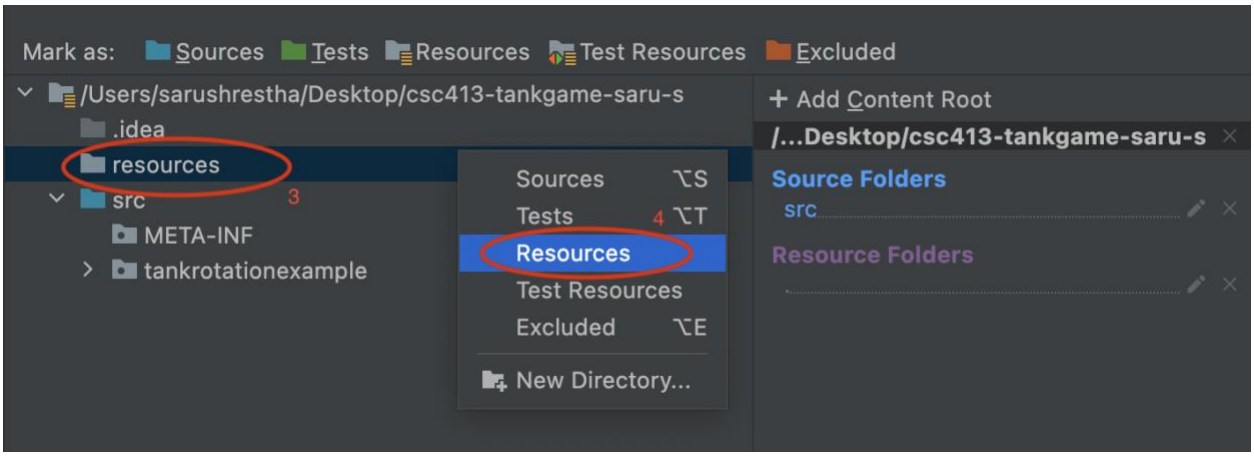


When the game is imported, the *src* folder will be added as root file by the IDE itself. However, it is our responsibility to add *Resources* folder as the root to access resources for the game. To set this up, we go to File > Project Structure > Modules > Resources.

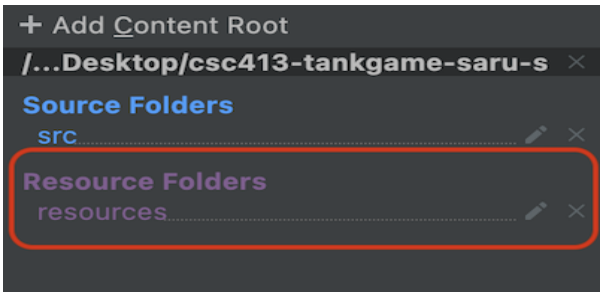

File
Edit
View
Navigate
New
Open...
Open Recent
Remote Development...
Close Project
Project Structure... ⌘ ;
File Properties
Local History
Save All ⌘S
Reload All from Disk ⌥⌘Y
Repair IDE
Invalidate Caches...
Manage IDE Settings
New Projects Setup
Save File as Template...
Export
Print...
Power Save Mode

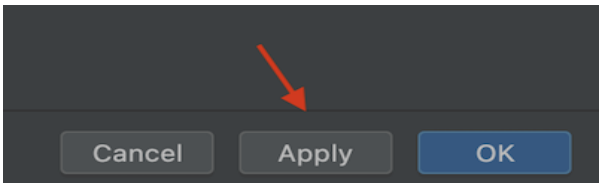

Project Settings
Project
Modules
Libraries
Facets
Artifacts
Platform Settings
SDKs
Global Libraries
Problems
csc413-tankgame-saru-
Name:
csc413-tankgame-saru-s
Sources
Paths
Dependencies
Language level:
Project default (18 - No new language features)
Mark as:
Sources
Tests
Resources
Test Resources
Excluded
/Users/sarushrestha/Desktop/csc413-tankgame-saru-s
.idea
resources
src
META-INF
tankrotationexample
Add Content Root
/...Desktop/csc413-tankgame-saru-s
Source Folders
src

Then, right click on *resources* folder then select *Resources* from the option. This will add the folder as resources root.

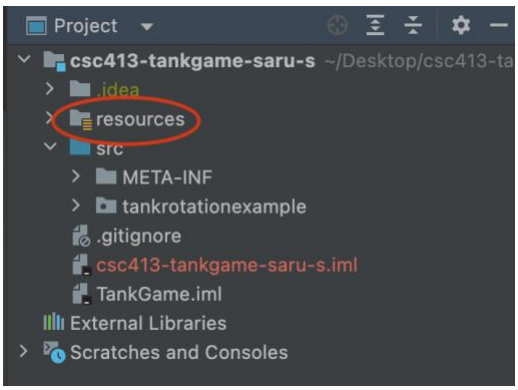


We can confirm it by checking in the panel on the side. Then, select apply.





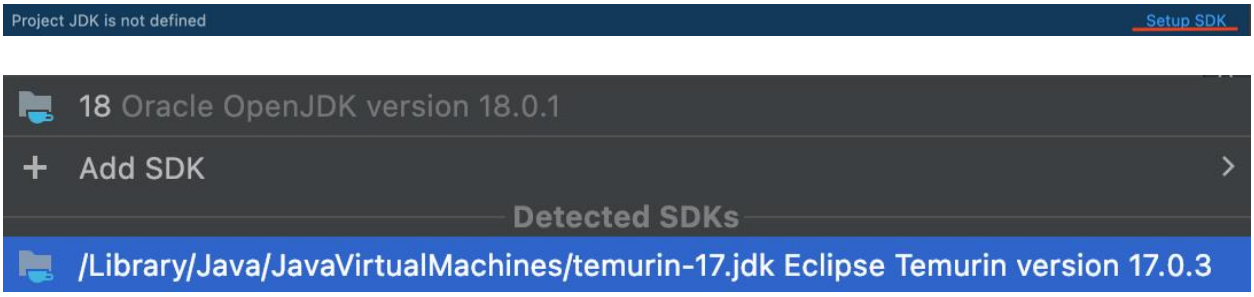
After setting resources root, the file structure is updated.



We must download JDK first to run those files. Most IDE gives us following notification suggesting to download JDK.



On the notification, we can select “Setup SDK”, then download the desired Java Development Kit.

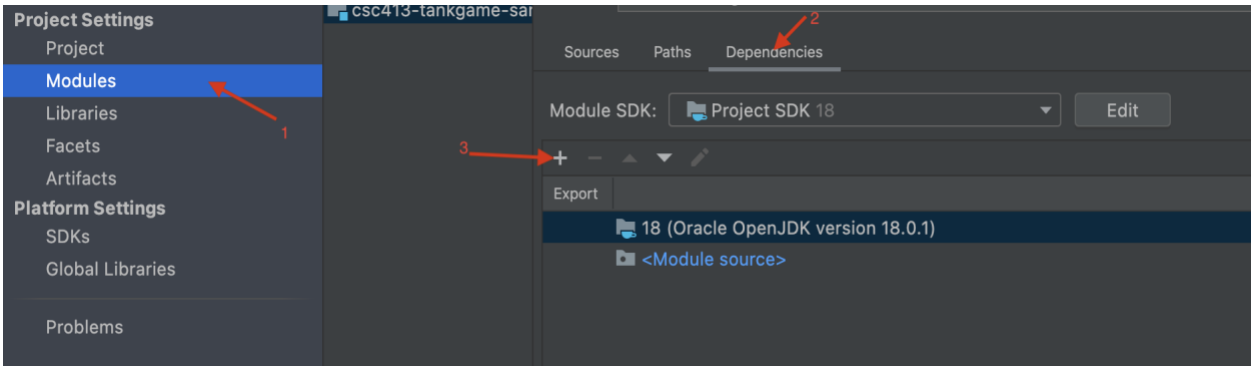


Then, the next step would be to create JAR file because we want to run the program in JAR.

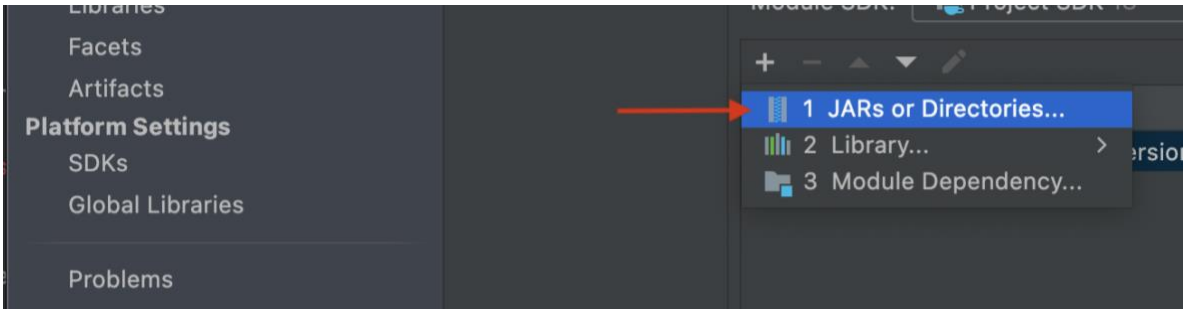
However, due to time restriction, JAR file was created using the help of IDE.

To create JAR file, go to File > Project Structure > Modules > Dependencies.

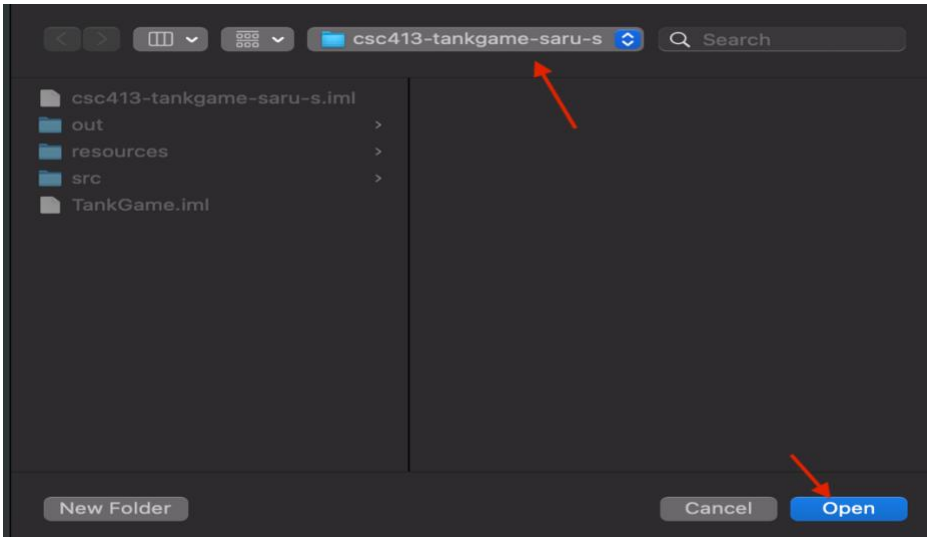
Then, click on ‘+’ button to add file.

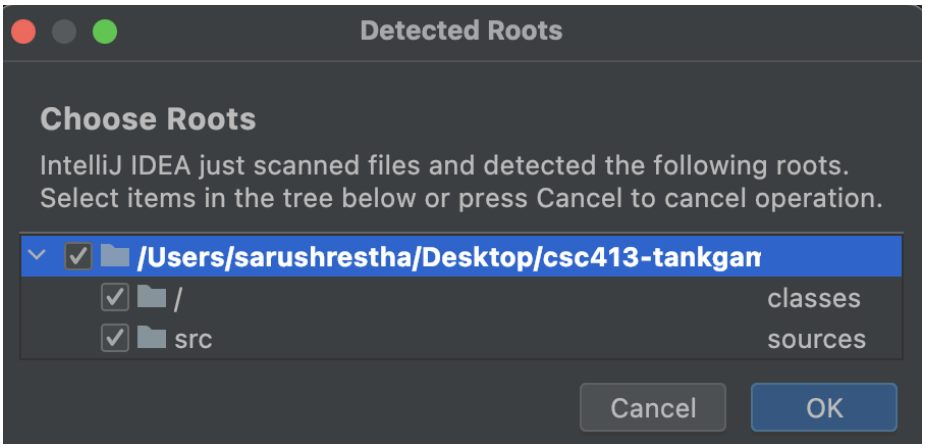


Select *JARs or Directories…*

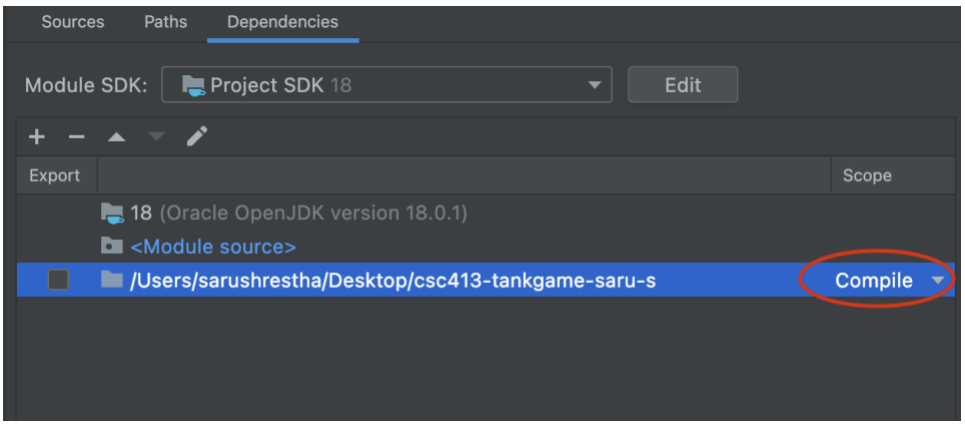


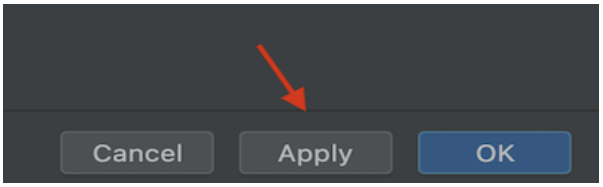
Select the file to be your JAR file. The recommendation would the current folder.



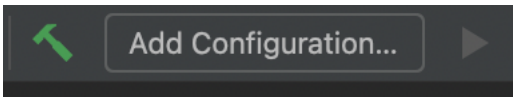


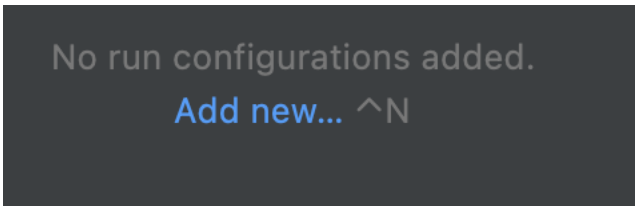
Then, make sure it is saved as *compile.* Then, click *Apply.*



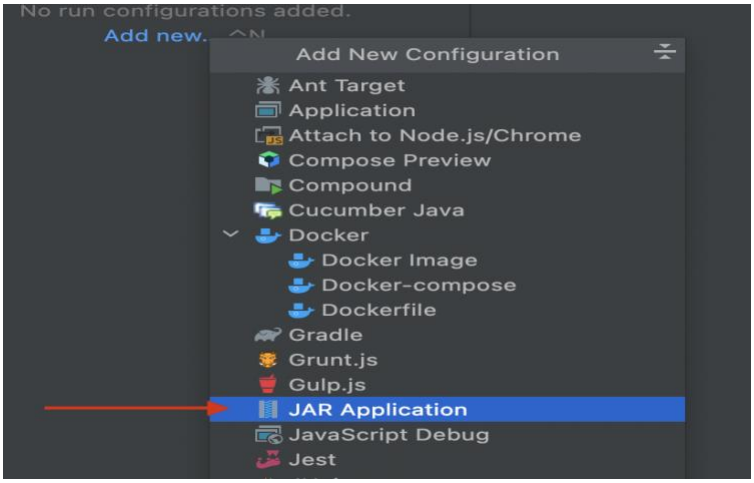


Then, we add JAR file to our *Run Configuration.*

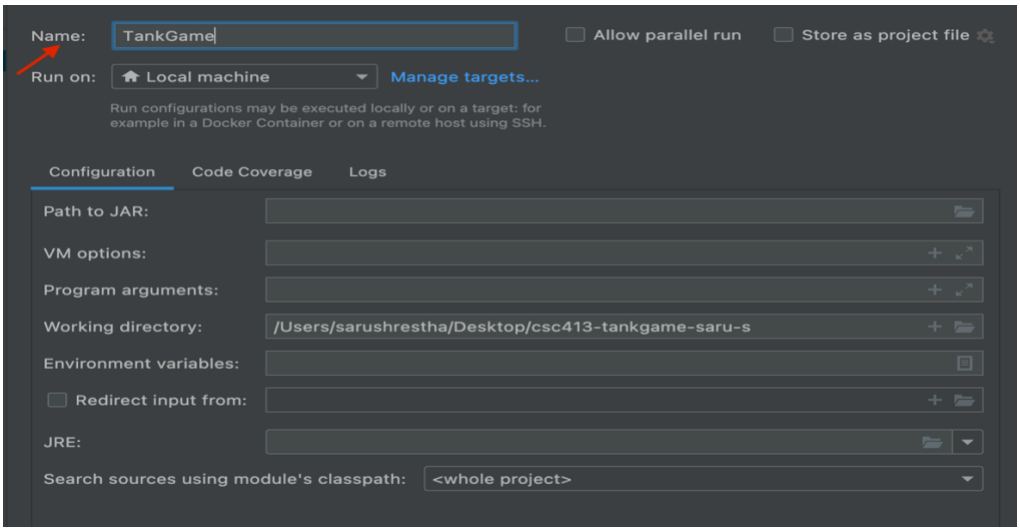




Select *Add new…* Then select JAR Application.



Then, the IDE will present the detected JAR file, select the *untitled* JAR, and give it a desired name.



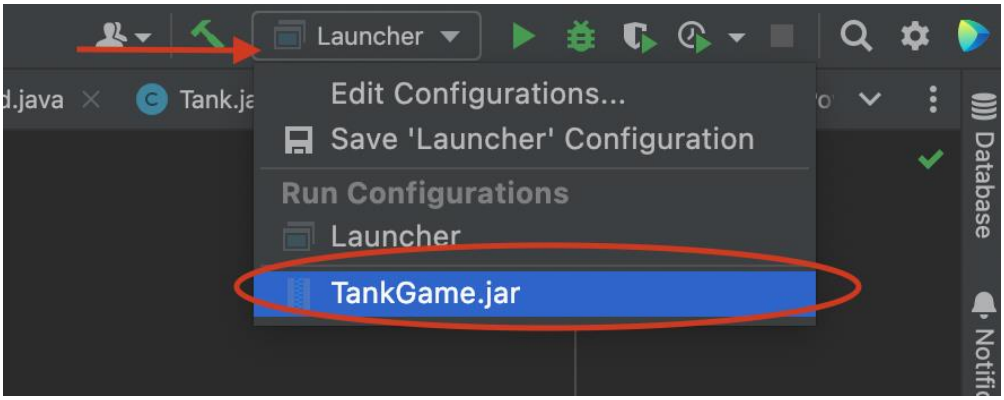
Then check in *Run Configuration.*



How to run your game.

Once the JAR files are added to the program, it is set as the configuration to run the program. To set the file as Run Configuration, the following steps can be done:

In the *Run Configuration* option on the tab, drag down the options, then select the JAR file saved while the file was added to the program. In this case, the JAR file name is *TankGame.jar.*



Then, click the green triangle button to run the program.

Assumption

Before building the game, assumptions were made regarding the program. First assumption was creating JAR file. Due to presence of limited time, it was assumed that JAR file would be created using the help of IDE.

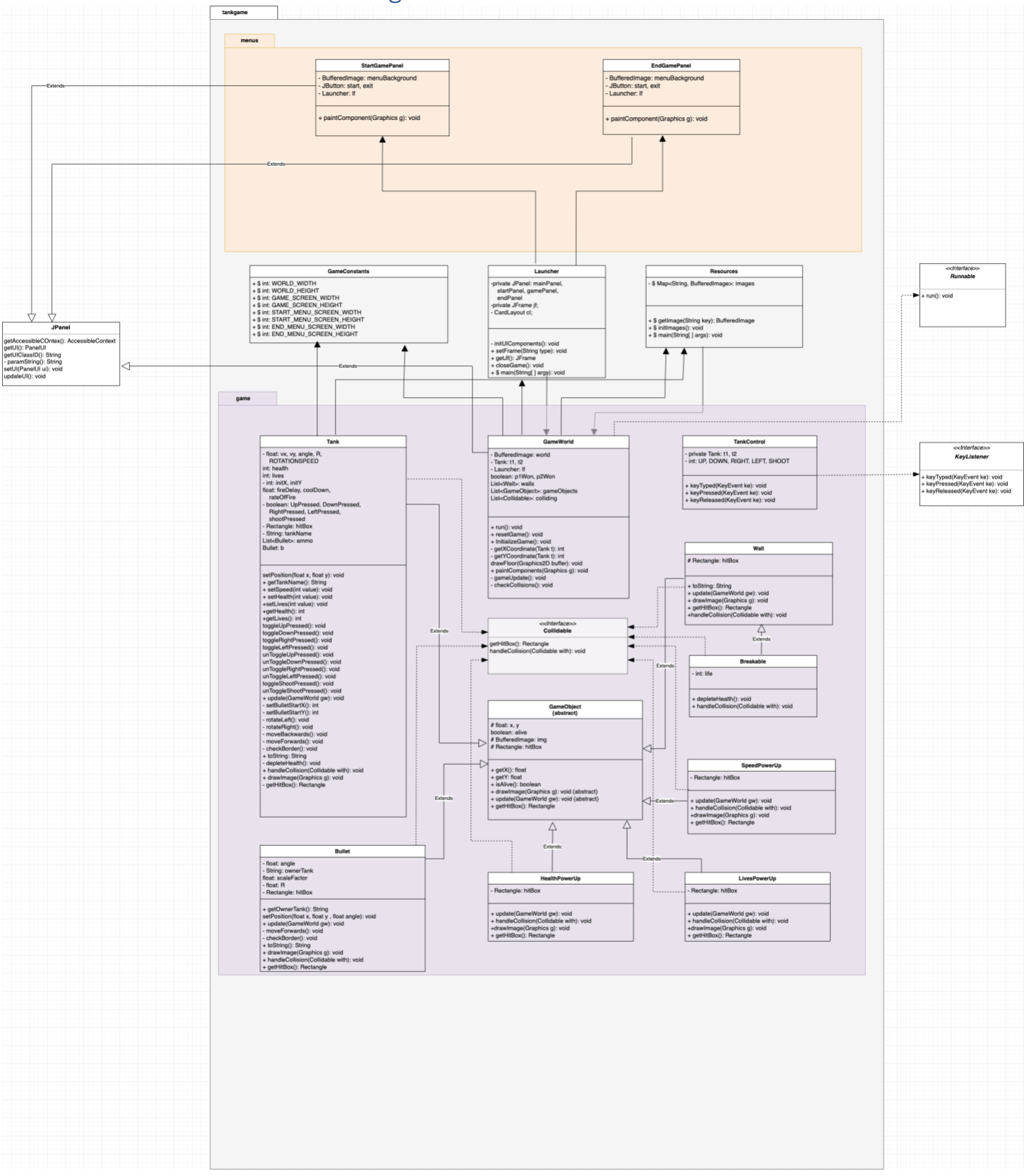
Second assumption made was regarding game objects. The game would be based on the Last-Man-Standing concept. There would be two players. The game would be played in local environment with players using the same device to control the game. The game would be simple with handful of game objects such as tanks, bullets, walls and few power boosts options. Another assumption concerning game objects was that the game objects would collide with each other.

Third assumption made about the program was the prediction of exceptions. The errors or exceptions are expected especially while reading or using the resources, while running the game, while using inheritance, interfaces, and polymorphism. The plan to tackle those exceptions are by using try-catch exception handling methods.

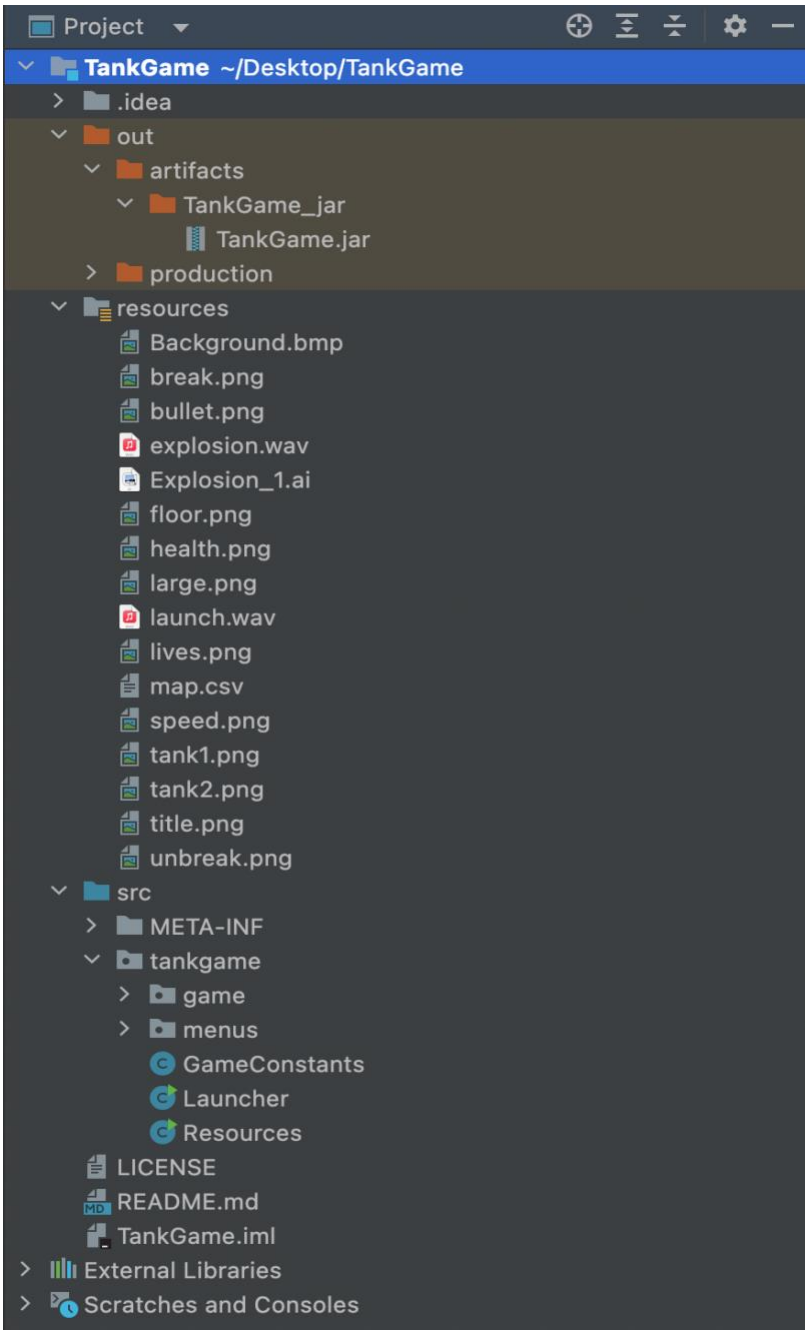
Fourth assumption was even after implementing all necessary components, the game might not perform as expected.

The last assumption was the issue with memory optimization. Since, this is one of the big projects I have built with an experience level as a beginner, I believe there would rooms for improvements in regard to using codes that would be helpful in memory optimization.

Tank Wars Game Class Diagram



File Structure of the Program



Description of Tank Wars Game Specific Classes

Launcher Class

Launcher Class launches the game. It works as a game initializer. This class also provides the GUI components to set up the layout for the game using JFrame. This class declares four panels which would be used throughout the different stages of the game:

- ⇒ *mainPanel* – This the main panel in JFrame. The layout of this panel is card layout allowing us to switch to sub-panels depending on game state.
- ⇒ *startPanel* – This panel displays the start menu. The menu contains two buttons: *Start* and *Exit.*
- ⇒ *gamePanel* – This panel displays the game screen. All the game objects are drawn and redrawn in this panel. The game also loops inside this panel.
- ⇒ *endPanel* – This panel displays the end menu. The menu contains two buttons: *Restart* and *Exit.*

StartGamePanel Class

StartGamePanel Class extends the *Launcher* Class. This class displays the Start Menu of the game. Therefore, this is the first class that user sees when running the game. The menu offers two options: *Start* and *Exit. Start* button starts the game and the *Exit* button simply quits the game. This class uses properties from *Launcher* class to start or exit the game. The game components are drawn/painted using GUI interface.

EndGamePanel Class

EndGamePanel Class extends the *Launcher* Class. This class is similar to *StartGamePanel* class while differing in one aspect i.e., this class is displayed at the end of the game. Thereupon, this class displays End Menu of the game. The menu offers two options: Re*start* and *Exit.* The *Restart* button starts again as desired by the players and *Exit* button closes the game screen.

This class uses properties from *Launcher* class to start or exit the game. The game components are drawn/painted using GUI interface.

GameConstants Class

GameConstants Class contains all the data fields that would be constants in the whole program. The data fields are: height and width of game world, game screen, start menu and end menu. The data field are made accessible to the whole program by defining as public static final variables.

Resources Class

Resources Class consists of all the resources that would be used in the program. The resources could include images, sounds, animations, text files and so on. The *Resources* class implements a static method and makes the saved resources available everywhere in the program. The reason for creating this class is to ease the programmers while using resources. The files filled in this class can be accessed in other classes just by using keywords assigned in this class instead of using file path every time we use those resources.

Collidable Interface

Collidable is an interface instead of class. The reason for creating *Collidable* interface is because contains abstract methods which can be implemented by other classes. Furthermore, creating an interface also supports multiple inheritances in JAVA. In *Collidable* interface, there are two methods which would be implemented by the game objects that are anticipated to collide with each other. This interface contains two methods:

- ⇒ Rectangle getHitBox – gets the width and height bounds of the game objects.
- ⇒ void handleCollision(Collidable with) – method to handle collision between two game objects.

GameObject Class

GameObject Class contains data fields and methods that are common in all the game objects. The data fields include float variables, BufferedImage variable. The method includes getter and setter for variables, method to confirm if the game object still exists in the game world, methods to draw image of game objects, to update any changes in game objects and to get height and width bounds of the game objects.

Tank Class

Tank Class contains all the data members and methods essential for the game object: Tank. *Tank* Class inherits from *GameObject* Class and implements methods from *Collidable* interface. This class incorporates the properties of tank such as movement of tank (forwards, backwards, rotation), position of tank. Besides that, it also contains required data fields and methods for other features of tank such as speed, health and live of tank. *Tank* class also implements shooting bullets from tank. In this class, the collision between tank object and other game objects like walls, bullets, power boosts are handled as well. The tank is drawn using drawImage method. To be noted, since there are two players tanks, this class is designed to be applicable for both tank-players. This class also updates the changes in tank object.

TankControl Class

TankControl Class is an event handler class. This class implements *KeyListener* and triggers events when users press or release the specified key. In this game, tank is the only game object that the user needs to control, therefore, this class contains all the fundamental keys controls required for the game.

Wall Class

Wall Class contains all the properties and functionality for the game object: wall. *Wall* Class extends *GameObject* Class and implements *Collidable* interface. In this game there are two types of walls: breakable and unbreakable. Breakable walls are the kinds that the players can

break by shooting their bullets at it. On the other hand, unbreakable walls are the kind that does not break even when force is used on it. *Wall* class id build for the game object wall, however more specifically for unbreakable wall. This class gets the dimension bounds of the wall and draws image of wall in the game. If there are any updates on wall object, this class updates it as well.

Breakable Class

Breakable Class contains the features for the game object: wall, to be specific, breakable walls. *Breakable* Class inherits from *Wall* class. Apart from inherited data fields and methods, this class handles the collision between breakable wall and Bullet. When the bullet is shot at this wall twice, the wall disappears making the space clear to promote further motion of tanks.

Bullet Class

Bullet Class contains information for the game object: bullet. This class extends *GameObject* Class and implements *Collidable* interface. This class contains movement of bullet (forwards). The class also updates any changes in the game object bullet. This class handles the collision between bullet and all the other game objects. Whenever the bullet hits any other object, the bullet disappears from the screen. This class also draws bullet on the game screen.

Power Ups

In this game, there are three power boosts available to the users. The powers boost speed, health, and lives count of tanks.

SpeedPowerUp Class

SpeedPowerUp Class is for speed boost of the game object tank. This class extends *GameObject* Class and implements *Collidable* interface. When tank encounters speed boost object, 3 units is added to the current speed of the tank.

HealthPowerUp Class

HealthPowerUp Class is for health boost of the game object tank. This class extends *GameObject* Class and implements *Collidable* interface. When tank intersects health boost object, the health of the tank resets to maximum health i.e., 100.

LivesPowerUp Class

LivesPowerUp Class is for lives count boost of the game object tank. This class extends *GameObject* Class and implements *Collidable* interface. When tank meets live boost object, the life of the tank is reset to maximum lives count i.e., 3.

GameWorld Class

GameWorld Class is where all the fragmented pieces come together. This class extends *JPanel* and implements *Runnable* interface. This class puts together all the game objects and their respective functionality, with the addition of more data members and functions to run the program, update the game objects, reset program, load resources, draw required objects. This class displays the game world/ game screen that the users see.

The game is initialized or ended in this class. The keys for tanks are defined in this class. In this game the designated keys are W, A, S, D, Spacebar, Up arrow, Down arrow, Left arrow, Right arrow and Enter keys.

The game layout is set in this class as well. The file with csv extension is read in this class, then decoded to represent the game objects. In this game, the csv file with numbers is used where those numbers are intended to represent game objects. For instance, 3,9 represents unbreakable walls, 2 is breakable walls, 4 is speed boost, 5 is health boost, and 6 is lives boost.

This class also sets the game screen layout as split screen where there are two screens each tracking the location of respective tanks. There is also mini map drawn in this class where users can view the whole game world at a glance.

GameWorld Class also checks if two game objects have collided with each other and calls handleCollision method which would carry out essential steps in occurrence of collision.

Self-Reflection on Software Design, Development and Reusability

In my perspective, this project was challenging yet rewarding. The reason I felt this project was challenging because this was a big software development project and my first experience using JAVA JAR. Because of fleeting time, the project soon became distressing. However, beside all the crazy turmoil, I see this project on positive light and take it as rewarding because this project truly is a learning experience. There are a lot to learn from this project like brainstorming ideas beforehand, time management, being open to ideas, researching resources, learning new codes and implementing all codes.

With respect to software design, development and reusability, this project contributed in showing the glimpse of software design. I have come to an understanding that software design is not only about writing codes. There are numerous steps before a coder sits down to write their code. The planning, collecting variables, assuming possible problems we might run into, diagrams, estimating time, and then writing code. This cycle keeps repeating throughout the whole development and there is an important factor, time that a developer should keep in check. I also realized the vital point of software development, memory optimization. Using Object-Oriented Programming and implementing encapsulation, polymorphism, inheritance aids a lot in making the code readable, efficient, and effective.

Conclusion

This project was an emotional rollercoaster. It was difficult at times but when there was visible result, it was exciting as well. I attempted to fulfill all the requirements listed in the prompt given regarding this project. Nonetheless, I suspect that not all features meet their expectations. There are times when certain functionality works and sometimes, they do not. One thing I would like to highlight is that when restarting the game, the previous winner tank

can still be seen in the new game set. This means that the restarted game, there would be three tanks. This is a bug which I plan to fix in the near future. In conclusion, this project was a valuable experience.